SmoothL1Loss#

https://pytorch.org/docs/stable/generated/torch.nn.SmoothL1Loss.html#torch.r

Description:

Creates a criterion that uses a squared term if the absolute element-wise error falls below beta and an L1 term otherwise. It is less sensitive to outliers than torch.nn.MSELoss and in some cases prevents exploding gradients (e.g. see the paper Fast R-CNN by Ross Girshick). For a batch of size NNN, the unreduced loss can be described as: If reduction is not none, then:

Function Signature

```
class torch.nn.SmoothL1Loss(size_average=None, reduce=None,
reduction='mean', beta=1.0)[source]#
Parameters
Shape:
forward(input, target)[source]#
Return type
```

Returns:

Tensor

Notes & Warnings

NOTE

Note Smooth L1 loss can be seen as exactly L1Loss, but with the ∣x−y∣<beta|x - y| < beta∣x−y∣<beta portion replaced with a quadratic function such that its slope is 1 at ∣x−y∣=beta|x - y| = beta∣x−y∣=beta. The quadratic segment smooths the L1 loss near ∣x−y∣=0|x - y| = 0∣x−y∣=0.

NOTE

Note Smooth L1 loss is closely related to HuberLoss, being equivalent to huber(x,y)/betahuber(x, y) / betahuber(x,y)/beta (note that Smooth L1's beta hyper-parameter is also known as delta for Huber). This leads to the following differences: As beta -> 0, Smooth L1 loss converges to L1Loss, while HuberLoss converges to a constant 0 loss. When beta is 0, Smooth L1 loss is equivalent to L1 loss. As beta -> +∞+\infty+∞, Smooth L1 loss converges to a constant 0 loss, while HuberLoss converges to MSELoss. For Smooth L1 loss, as beta varies, the L1 segment of the loss has a constant slope of 1. For HuberLoss, the slope of the L1 segment is beta.

Detailed Documentation

Documentation

Creates a criterion that uses a squared term if the absolute element-wise error falls below beta and an L1 term otherwise. It is less sensitive to outliers than torch.nn.MSELoss and in some cases prevents exploding gradients (e.g. see the paper Fast R-CNN by Ross Girshick).

For a batch of size NNN, the unreduced loss can be described as:

If reduction is not none, then:

Smooth L1 loss can be seen as exactly L1Loss, but with the ∣x−y∣<beta|x - y| < beta ∣x−y∣<beta portion replaced with a quadratic function such that its slope is 1 at ∣x−y∣=beta|x - y| = beta ∣x−y∣=beta. The quadratic segment smooths the L1 loss near ∣x−y∣=0|x - y| = 0 ∣x−y∣=0.

Smooth L1 loss is closely related to HuberLoss, being equivalent to huber(x,y)/betahuber(x, y) / betahuber(x,y)/beta (note that Smooth L1's beta hyper-parameter is also known as delta for Huber). This leads to the following differences:

As beta -> 0, Smooth L1 loss converges to L1Loss, while HuberLoss converges to a constant 0 loss. When beta is 0, Smooth L1 loss is equivalent to L1 loss.

As beta -> +∞+\infty+∞, Smooth L1 loss converges to a constant 0 loss, while HuberLoss converges to MSELoss.

For Smooth L1 loss, as beta varies, the L1 segment of the loss has a constant slope of 1. For HuberLoss, the slope of the L1 segment is beta.

size_average (bool, optional) – Deprecated (see reduction). By default, the losses are averaged over each loss element in the batch. Note that for some losses, there are multiple elements per sample. If the field size_average is set to False, the losses are

instead summed for each minibatch. Ignored when reduce is False. Default: True

reduce (bool, optional) – Deprecated (see reduction). By default, the losses are averaged or summed over observations for each minibatch depending on size_average. When reduce is False, returns a loss per batch element instead and ignores size_average. Default: True

reduction (str, optional) – Specifies the reduction to apply to the output: 'none' | 'mean' | 'sum'. 'none': no reduction will be applied, 'mean': the sum of the output will be divided by the number of elements in the output, 'sum': the output will be summed. Note: size_average and reduce are in the process of being deprecated, and in the meantime, specifying either of those two args will override reduction. Default: 'mean'

beta (float, optional) – Specifies the threshold at which to change between L1 and L2 loss. The value must be non-negative. Default: 1.0

Input: $(*)(*)$, where $**$ means any number of dimensions.

Target: $(*)(*)$, same shape as the input.

Output: scalar. If reduction is 'none', then $(*)(*)$, same shape as the input.

Runs the forward pass.